

רמזים

שים לב

שלוש דרגות בכל קושי. קרא את הדחיפה, נסה עשר דקות, ורק אז את האסטרטגיה. הכמעט-פתרון הוא הרשת, לא הקיצור. יש כאן שנים עשר קשיים, והם מסודרים לפי הסדר שבו הם קורים בפועל — לא לפי סדר החלקים בברף.

א · "אני לא מצליח לראות אף אחת מארבע התקלות"

דחיפה. כל אחת מהארבע דורשת שתי פעולות בסדר מסוים. פעולה אחת לבדה תמיד נראית נכונה — זו בדיוק הסיבה שהיישום הזה עבר בדיוק ידנית של מי שכתב אותו.

אסטרטגיה. אחרי כל צעד, עצור וכתוב מה אתה מצפה לראות לפני שאתה מסתכל. תקלה 3 היא הכי מעניינת והכי קשה לראות, ולכן היא זו שכדאי לעשות לאט: חפש `כמו 1`, לחץ `+1`, ורק אז נקה את החיפוש.

כמעט-פתרון. שני מספרים ששווה להשוות בכל שלב — כמה פריטים יש בנתונים וכמה שורות יש על המסך. את הראשון אפשר לראות בקונסולה: פתח את הקונסולה, כתוב `console.table` על המערך, ותקבל טבלה. אם השניים לא מסכימים, מצאת תקלה בלי לקרוא שורת קוד אחת.

ב · "פירקתי לשלושה קבצים והעמוד ריק, בלי שגיאה"

דחיפה. ההרשמה של `render` היא רישום לשינויים הבאים. כמה שינויים היו מאז שהעמוד נטען?

אסטרטגיה. ארבע שורות ב-`app.js`, בסדר הזה: להכניס את הנתונים, `wire`, `subscribe`, ואז קריאה מפורשת אחת ל-`render` עם ה-`state` הנוכחי. אם השורה הרביעית לא שם, העמוד ריק עד שהמשתמש מקליד תו — וזה נראה בדיוק כמו `render` שבור.

כמעט-פתרון. בדוק בשתי שניות אם זו הבעיה: הקלד תו כלשהו בתיבת החיפוש. אם הרשימה מופיעה פתאום — הרינדור עובד מצוין, פשוט אף אחד לא ביקש ממנו לצייר בפעם הראשונה.

ג · "העמוד ריק, ובקונסולה יש שגיאה על מודול"

דחיפה. קרא את השם בשגיאה. מודול ES דורש את הסיומת בייבוא, והדפדפן לא ינחש אותה.

אסטרטגיה. שלוש בדיקות, בסדר הזה: הסיומת `.js`. נמצאת בכל `import`; הנתיב מתחיל ב-`./`; ואתה מריץ עם `Live Server` ולא בלחיצה כפולה. השלישית היא זו שתופסת אנשים שהעבירו קבצים ואז פתחו את הקובץ מהאצבע.

כמעט-פתרון. יש גם מקרה שקט יותר: ייבוא של שם שלא יוצא. אם `render.js` מייבא `selectVisible` ו-`state.js` מייצא `selectvisible`, הדפדפן אומר את זה במפורש בקונסולה ומצביע על השורה. חפש את המילה `export` בקובץ שממנו אתה מייבא וודא שהשם זהה בדיוק.

ד · "לא ברור לי מה נכנס ל-`state` ומה לא"

דחיפה. שאלה אחת לכל שדה: האם אני יכול לחשב אותו מהשדות האחרים? אם כן — הוא לא שייך לשם.

אסטרטגיה. עבור על מה שיש לך היום ותייג. האוסף עצמו: לא ניתן לחישוב, נכנס. מה שהמשתמש הקליד וכל מה שהוא בחר בפקדים: לא ניתן לחישוב, נכנס. הרשימה המסוננת, המונה, רשימת המדפים, כל מספר בעמוד הסיכום: **כולם פונקציות של הראשונים**, ולכן אף אחד מהם לא נכנס.

כמעט-פתרון. מבחן בשתי שניות לכל שדה חשוד: אם משהו ישנה את האוסף **וישכח** לעדכן את השדה הזה — האם המסך ישקר? אם התשובה כן, זו גזירה שמורה, וצריך למחוק אותה ולכתוב פונקציה במקומה.

ה · "החיפוש עובד אבל ההוספה במהלך חיפוש מתנהגת מוזר"

דחיפה. אם `visible` הוא שדה שמישהו מעדכן, יש עכשיו שני מקומות שצריכים לזכור לעדכן אותו. כמה מטפלים יש לך שמשנים את האוסף?

אסטרטגיה. אין `visible`. יש `state.items` ו-`state.query`, ויש **פונקציה** שמקבלת `state` ומחזירה מה שמוצג. אף מטפל לא קורא לה ואף מטפל לא שומר את התוצאה שלה — רק `render` קורא לה, בכל ציור, מחדש.

כמעט-פתרון. מטפל החיפוש שלך צריך להיות שורה אחת שאומרת "מה שהמשתמש הקליד הוא עכשיו הערך של השדה". זהו. אם יש בו `filter`, הסינון נמצא במקום הלא נכון.

ו · "המיון עובד ואז נעלם ברגע שמוסיפים פריט"

דחיפה. מה בדיוק שמר את סדר המיון? אם התשובה היא "השורות על המסך" — הן נמחקות בציור הבא.

אסטרטגיה. מיון הוא **גזירה**, בדיוק כמו סינון. הפקד שומר מה נבחר; הפונקציה שמחשבת מה מוצג מסתכלת בשדה הזה וממיינת. מטפל המיון הוא שורה אחת ואינו ממין כלום.

כמעט-פתרון. מיון **עותק**. סינון כבר מחזיר מערך חדש, אז אם אתה ממין את התוצאה של `filter` אתה בטוח. אם אתה ממין מערך שהגיע ישירות מ-`state`, אתה עורך את הנתונים שלך בכל פעם שמישהו נוגע בפקד תצוגה.

ז · "הסרה מסירה את הפריט הלא נכון אחרי סינון"

דחיפה. מה כתוב על השורה כדי לזהות אותה? אם זה מספר, שאל את עצמך מספר של מה — מקום ברשימה המוצגת, או מקום באוסף?

אסטרטגיה. מיקום הוא לא זהות. שורה זזה בכל מיון, בכל חיפוש ובכל סינון; הפריט לא משתנה. שים על ה-`<li>` את המזהה של הפריט, ובמטפל קרא אותו וחפש **לפי מזהה**.

כמעט-פתרון. שתי השורות שמשתנות: בבניית השורה, במקום לרשום את מקומה רשום את `item.id`; ובמטפל, במקום להיכנס למערך לפי מספר, השתמש ב-`filter` שמוציא את מי שהמזהה שלו שווה למה שקראת. שים לב שאין שום מקום שבו אתה צריך לדעת איפה הפריט היה.

ח · "הכפתורים עובדים ואז מפסיקים אחרי שאני מקליד בחיפוש"

דחיפה. `render` מרוקן את הרשימה בכל שינוי. איפה רשמת את המאזין?

אסטרטגיה. מאזין שנרשם על כפתור מת עם הכפתור, וכל הכפתורים נמחקים בכל ציור. רשום מאזין אחד על ה-`<ul>` — שהוא נקודת עיגון ש-`render` מרוקן אבל **לא מחליף** — ומצא בפנים על מה נלחץ.

כמעט-פתרון. יש גם וריאציה גרועה יותר של אותה טעות: רישום המאזינים **בתוך** `render`. זה עובד, ולכן מסוכן — אחרי עשר הקשות בתיבת החיפוש יש עשרה מאזינים על אותו כפתור, וכל לחיצה מריצה את המטפל עשר פעמים. `wire` נקראת **פעם אחת**, מ-`app.js`.

ט · "render" שלי צריך לדעת באיזה עמוד הוא

דחיפה. מה קורה כשאתה מחפש אלמנט שלא קיים בעמוד הזה? מה מחזיר `querySelector`?

אסטרטגיה. כל אזור הוא פונקציה קטנה שמתחילה באותן שתי שורות: לחפש את נקודת העיגון שלה, ולחזור מיד אם היא אינה שם. ואז `render` פשוט קוראת לכולן, וכל אחת מהן מחליטה בעצמה אם יש לה מה לעשות בעמוד הזה.

כמעט-פתרון. זה גם עונה על השאלה "איך מוסיפים עמוד רביעי?": מוסיפים לו נקודת עיגון ב-HTML, כותבים פונקציה קטנה, ומוסיפים שורה אחת ל-`render`. אף קובץ אחר לא משתנה, ואף `if` על שם העמוד לא נכתב בשום מקום.

י · "פקד הסינון מתאפס לי כל פעם שאני מקליד בחיפוש"

דחיפה. אתה בונה מחדש את האפשרויות בכל ציור — וציור קורה בכל הקשה. מה קורה לערך הנבחר כשמחליפים לאלמנט את כל הילדים שלו?

אסטרטגיה. קרא את הערך הנבחר לתוך משתנה **לפני** שאתה בונה מחדש, והחזר אותו אחרי. שתי שורות. זה **החריג היחיד** בכל השבוע לכלל "לא קוראים את המסך", והוא לא באמת חריג: אתה לא שואל את המסך מה נכון, אתה מחזיר לפקד דפדפן ערך שאיבד אותו כשהוחלפו ילדיו.

כמעט-פתרון. זה צעד 2 של מחזור 3, על `#genre`. ויש עוד מקרה קצה אחד: אם המדף שהיה נבחר נעלם — כי הסרת את הפריט האחרון בו — ההשמה בחזרה לא תופסת, והפקד נשאר **ריק**: `value` הוא מחרוזת ריקה ואף אפשרות לא מסומנת (נמדד ב-Chromium). שורה אחת נוספת — אם הערך ריק, "הכול" — מחזירה אותו לאפשרות שקיימת תמיד.

יא · "עמוד הסיכום מראה מספר אחר מעמוד הרשימה"

דחיפה. כמה מקומות בקוד שלך סופרים? ספור אותם.

אסטרטגיה. אם התשובה גדולה מאחד, זה לא באג שצריך לתקן — זו הסיבה שהמטלה מבקשת למחוק את `js/summary.js`. עמוד הסיכום לא סופר: פונקציה אחת ב-`state.js` מחזירה את כל מה שהעמוד מציג, ו-`render` כותבת אותו.

כמעט-פתרון. אחרי שמחקת את הקובץ, בדוק שהעמוד לא מנסה לטעון אותו: `summary.html` צריך לטעון את אותו סקריפט כמו שני העמודים האחרים. שגיאת 404 על קובץ שמחקת היא שקטה בעמוד ורועשת בלשונית Network.

יב · "הביטול שלי ארוך ואני לא בטוח מה לשמור"

דחיפה. אתה שומר את מה שהוסר, או את מה שהיה?

אסטרטגיה. שמור את **המערך** שהחלפת, שלם. הביטול הוא אז השמה אחת: המערך הישן חוזר להיות האוסף. שחזור של פריט בודד מחייב אותך לדעת גם איפה הוא היה, ואיפה הוא היה זו שאלה שאין לה תשובה אחרי מיון.

כמעט-פתרון. מבנה שעובד: שדה אחד שהוא או `null` או אובייקט קטן עם שני שדות — המערך הקודם, ושם הפריט שהוסר כדי שאפשר יהיה לכתוב משפט. המטפל של ההסרה ממלא אותו, `render` מצייר ממנו כפתור, המטפל של הכפתור מחזיר את המערך ומאפס אותו ל-`null`. **אם זה יותר מארבע שורות, אתה משחזר במקום להחזיר.**